Instituto Politécnico Nacional

Escuela Superior de Cómputo

Ingeniería en Sistemas Computacionales

Programa Tablero

Alumno: Linares Ojeda Carlos Elias

No. de Boleta: 2024630337

Grupo: 4CM2

Unidad de Aprendizaje: Teoría de la Computación

Profesor: Genaro Juárez Martínez



Ciudad de México, 7 de abril de 2025

Índice

1 Introducción **2**

2 Marco teórico **2**
2.1 Autómata Finito No Determinístico (AFND) 2
2.2 Representación del tablero 3
2.3 Alfabeto de movimientos 3
2.4 Caminos y estados 4
2.5 Condición necesaria para ganar 5

3 Desarrollo **5**
3.1 Objetivo y alcance de la versión actual 5
3.2 Planteamiento 6
3.3 Estructura del programa 6
3.4 Interfaz gráfica 7
3.5 Explicación de los algoritmos utilizados 9
3.5.1 Función de transición 9
3.5.2 Cálculo de todos los caminos del AFND 9
3.5.3 Generación de los archivos de caminos 10
3.5.4 Selección de rutas y del turno inicial 10
3.5.5 Avance de la simulación y manejo de colisiones 11
3.5.6 Redirección de jugadores 12
3.5.7 Fin de la partida 13
3.5.8 Validación de las entradas 13
3.5.9 Animación de los movimientos 14
3.6 Uso del programa 14
3.7 Particularidades del funcionamiento actual 14

4 Resultados **15**
4.1 Inicio de una partida 15
4.2 Victoria del Jugador 2 15
4.3 Colisión y cesión del paso 16
4.4 Redirección y victoria del Jugador 1 17
4.5 Archivos de caminos generados 19
4.6 Crecimiento del número de caminos 20

5 Conclusión **22**

6 Referencias bibliográficas **22**

1 Introducción

Un autómata finito no determinístico es aquel que puede estar en varios estados a la vez. Al igual que un autómata finito determinístico, tiene un conjunto finito de estados, un conjunto finito de símbolos de entrada, un estado inicial y un conjunto de estados finales. También tiene una función de transición.

Un autómata finito no determinístico se representa esencialmente como un autómata finito determinístico [1]:

$$A = (Q, \Sigma, \delta, q_0, F) \tag{1}$$

donde:

- Q es un conjunto finito de estados.
- Σ es un conjunto finito de símbolos de entrada.
- q_0, un elemento de Q, es el estado inicial.
- F, un subconjunto de Q, es el conjunto de estados finales (o de aceptación).
- δ, la función de transición, es una función que toma como argumentos un estado de Q y un símbolo de entrada de Σ y devuelve un subconjunto de Q. Observe que la única diferencia entre un AFN y un AFD se encuentra en el tipo de valor que devuelve δ: un conjunto de estados en el caso de un AFN y un único estado en el caso de un AFD.

En el contexto de este trabajo, aplicaremos los conceptos de autómatas finitos no determinísticos para resolver un problema de movimiento en un tablero de ajedrez modificado. La simulación nos permitirá visualizar cómo dos piezas pueden recorrer diferentes caminos siguiendo reglas específicas basadas en movimientos ortogonales y diagonales. El programa, escrito en Java, calcula todos los caminos que puede seguir el autómata de cada pieza, los guarda en archivos de texto y simula una partida entre dos jugadores en una interfaz gráfica, en la que se observan los turnos, las colisiones y al ganador.

2 Marco teórico

Para entender el funcionamiento del programa, es esencial comprender los siguientes conceptos:

2.1 Autómata Finito No Determinístico (AFND)

En un AFND, cada estado puede tener múltiples transiciones para un mismo símbolo de entrada, o incluso ninguna. Esto contrasta con los autómatas deterministicos, donde cada símbolo lleva a un único estado siguiente. Esta característica de no determinismo nos permite modelar sistemas donde pueden existir múltiples posibilidades de evolución.

Para procesar una cadena completa se utiliza la función de transición extendida $\hat{\delta}$, que devuelve el conjunto de estados en los que puede encontrarse el autómata después de leer una cadena w. Se define por inducción sobre la longitud de la cadena:

$$\hat{\delta}(q, \varepsilon) = \{q\}, \qquad \hat{\delta}(q, wa) = \bigcup_{p \in \hat{\delta}(q,w)} \delta(p, a). \tag{2}$$

El autómata acepta la cadena w si al menos uno de esos estados es final, es decir, si $\hat{\delta}(q_0, w) \cap F \neq \emptyset$. Otra forma de verlo es que, en cada paso, el autómata prueba todas las transiciones posibles al mismo tiempo: cada sucesión de elecciones es un camino distinto, y todos los caminos juntos forman el *árbol de cómputo* de la cadena. La cadena se acepta si al menos uno de esos caminos termina en un estado final.

2.2 Representación del tablero

El programa utiliza un tablero de 4×4 casillas, numeradas del 1 al 16 de izquierda a derecha y de arriba a abajo (Figura 1). Si las filas y las columnas se cuentan desde 0, la fila y la columna de la casilla q son

$$\text{fila}(q) = \left\lfloor \frac{q-1}{4} \right\rfloor, \qquad \text{col}(q) = (q-1) \text{ mód } 4, \tag{3}$$

y, de manera inversa, $q = 4 \cdot \text{fila}(q) + \text{col}(q) + 1$. El programa utiliza estas fórmulas tanto para calcular los movimientos como para dibujar las piezas en la ventana.

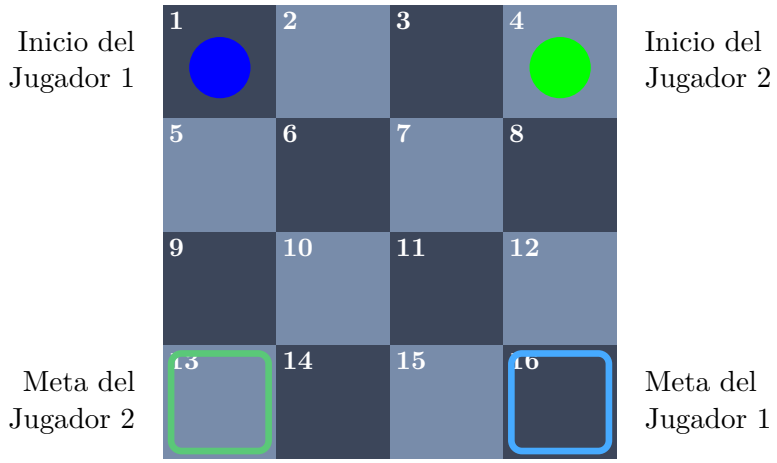


Figura 1: Tablero de 4×4 con la numeración de las casillas, la posición inicial de cada jugador y sus casillas meta, marcadas con un recuadro del color del jugador, igual que en el programa.

En este tablero, dos jugadores realizarán movimientos siguiendo reglas específicas (en adelante, J1 y J2 denotan al Jugador 1 y al Jugador 2):

- Jugador 1 (ficha azul): comienza en la posición 1 y debe llegar a la posición 16.
- Jugador 2 (ficha verde): comienza en la posición 4 y debe llegar a la posición 13.

Las dos piezas parten de esquinas vecinas y sus metas están en las esquinas opuestas, por lo que es frecuente que sus trayectorias se crucen en las casillas centrales del tablero.

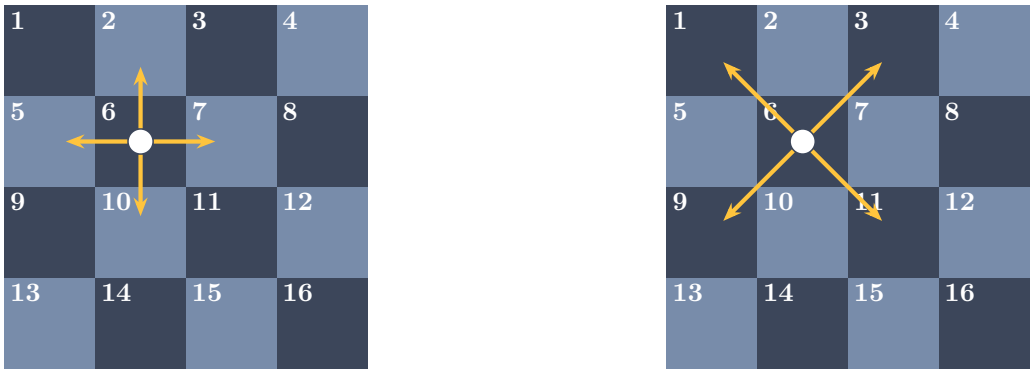
2.3 Alfabeto de movimientos

Los movimientos se definen mediante dos símbolos, es decir, $\Sigma = \{r, b\}$:

- r: representa un movimiento ortogonal, a una casilla adyacente en horizontal o en vertical (como una torre que solo avanza una casilla).

- b: representa un movimiento diagonal, a una casilla adyacente en diagonal (como un alfil que solo avanza una casilla).

Este problema es una adaptación del ejemplo del tablero de ajedrez de las láminas del curso de autómatas de Stanford [2]. Una cadena de entrada, por ejemplo $w = rbbr$, indica el *tipo* de movimiento que debe hacer la pieza en cada paso, pero no la *dirección*: como desde una casilla casi siempre hay varias casillas posibles, la pieza puede seguir muchos caminos distintos con la misma cadena. Ahí está el no determinismo del problema. La Figura 2 muestra los movimientos posibles desde la casilla 6.



(a) Con r: $\delta(6, r) = \{2, 5, 7, 10\}$. **(b)** Con b: $\delta(6, b) = \{1, 3, 9, 11\}$.

Figura 2: Movimientos posibles desde la casilla 6 con cada símbolo del alfabeto.

2.4 Caminos y estados

En términos de autómatas, cada casilla del tablero representa un estado, y cada movimiento (r o b) es un símbolo de entrada. Cada jugador se modela como un AFND; ambos comparten los estados, el alfabeto y la función de transición, y solo difieren en su estado inicial y su estado final:

$$A_1 = (Q, \Sigma, \delta, 1, \{16\}), \qquad A_2 = (Q, \Sigma, \delta, 4, \{13\}), \tag{4}$$

con $Q = \{1, 2, \ldots, 16\}$ y $\Sigma = \{r, b\}$.

La función de transición δ determina, para cada estado y símbolo, el conjunto de estados posibles a los que se puede transitar. En este caso depende de la posición actual y del tipo de movimiento:

$$\delta(q, r) = \{\, p \in Q : |\,\text{fila}(p) - \text{fila}(q)| + |\,\text{col}(p) - \text{col}(q)| = 1 \,\}, \tag{5}$$

$$\delta(q, b) = \{\, p \in Q : |\,\text{fila}(p) - \text{fila}(q)| = 1 \text{ y } |\,\text{col}(p) - \text{col}(q)| = 1 \,\}. \tag{6}$$

La Tabla 1 muestra la función completa. Las esquinas tienen pocas opciones (dos con r y una sola con b), mientras que las cuatro casillas centrales tienen cuatro opciones con cada símbolo. Además, $\delta(q, a)$ nunca es vacío: toda casilla tiene al menos una vecina ortogonal y una diagonal, así que para cualquier cadena válida existe al menos un camino del autómata. En la partida, una pieza sí puede permanecer en su lugar cuando debe ceder el paso por una colisión.

Tabla 1: Función de transición δ del AFND.

q	$\delta(q,r)$	$\delta(q,b)$	q	$\delta(q,r)$	$\delta(q,b)$
1	$\{2,5\}$	$\{6\}$	9	$\{5,10,13\}$	$\{6,14\}$
2	$\{1,3,6\}$	$\{5,7\}$	10	$\{6,9,11,14\}$	$\{5,7,13,15\}$
3	$\{2,4,7\}$	$\{6,8\}$	11	$\{7,10,12,15\}$	$\{6,8,14,16\}$
4	$\{3,8\}$	$\{7\}$	12	$\{8,11,16\}$	$\{7,15\}$
5	$\{1,6,9\}$	$\{2,10\}$	13	$\{9,14\}$	$\{10\}$
6	$\{2,5,7,10\}$	$\{1,3,9,11\}$	14	$\{10,13,15\}$	$\{9,11\}$
7	$\{3,6,8,11\}$	$\{2,4,10,12\}$	15	$\{11,14,16\}$	$\{10,12\}$
8	$\{4,7,12\}$	$\{3,11\}$	16	$\{12,15\}$	$\{11\}$

Un *camino* para la cadena $w = a_1 a_2 \cdots a_n$ es una sucesión de estados $q_0, q_1, \ldots, q_n$ en la que q_0 es el estado inicial del jugador y $q_i \in \delta(q_{i-1}, a_i)$ para $i = 1, \ldots, n$. Cada camino tiene $n+1$ estados: el inicial y uno por cada símbolo leído. Un camino es *ganador* si además termina en el estado final, es decir, si $q_n \in F$. Por lo tanto, la cadena w es aceptada por el autómata de un jugador si y solo si tiene al menos un camino ganador.

2.5 Condición necesaria para ganar

Si se colorea el tablero como uno de ajedrez, según la paridad de $\text{fila}(q) + \text{col}(q)$, un movimiento r cambia exactamente una de las dos coordenadas en una unidad, así que siempre lleva a una casilla del otro color; en cambio, un movimiento b cambia ambas coordenadas y conserva el color. Las casillas inicial y final de cada jugador son del mismo color (1 y 16 tienen fila + col par; 4 y 13, impar), de modo que:

> *una cadena solo puede tener caminos ganadores si contiene un número par de símbolos r.*

Es una condición necesaria, pero no suficiente: la cadena rrb tiene dos símbolos r y, aun así, ninguno de sus 14 caminos llega a la casilla 16. Además, como cada meta está a tres filas y tres columnas de distancia del inicio, se necesitan al menos tres movimientos para ganar: la cadena más corta con la que el Jugador 1 puede ganar es *bbb* ($1 \to 6 \to 11 \to 16$), y lo mismo ocurre con el Jugador 2 ($4 \to 7 \to 10 \to 13$). En la práctica, alrededor de la mitad de las cadenas aleatorias tiene un número impar de símbolos r y, por lo tanto, ningún camino ganador; esto se observa con frecuencia en los resultados (Sección 4).

3 Desarrollo

3.1 Objetivo y alcance de la versión actual

El programa permite observar cómo una misma cadena de movimientos produce varias rutas en un tablero de 4×4. Calcula los caminos de dos autómatas, identifica cuáles terminan en sus estados de aceptación y utiliza una ruta por jugador para representar una partida con prioridad de turno, colisiones y un resultado final.

La versión actual ofrece entrada manual de las dos cadenas y generación aleatoria en modo automático. Ambas modalidades se controlan desde la ventana. El modo manual admite entre 1 y 12 símbolos por jugador; el automático permite elegir una longitud entre

3 y 12. La generación aleatoria se realiza al iniciar una partida automática y no exige que las cadenas tengan caminos ganadores.

La ventana muestra el tablero y las rutas que siguen las fichas. Los demás caminos quedan registrados en cuatro archivos de texto; no hay un visor de la red completa de transiciones en ejecución. El árbol de cómputo incluido en este reporte es un diagrama explicativo del cálculo del autómata.

3.2 Planteamiento

El programa simula un juego en un tablero de ajedrez de 4×4 donde dos jugadores intentan llegar a sus respectivas posiciones finales siguiendo cadenas de movimientos formadas por los símbolos r y b. Se utiliza un enfoque basado en autómatas finitos no determinísticos para calcular todos los posibles caminos que cada jugador puede tomar. Todo se controla desde una interfaz gráfica, sin necesidad de escribir en la consola.

El funcionamiento general del programa es el siguiente:

1. En el panel de control, el usuario elige el modo *Manual* o *Automático*.
2. En el modo manual escribe la cadena de cada jugador; en el modo automático elige la longitud (de 3 a 12 símbolos) y el programa genera al azar las dos cadenas, con la misma longitud.
3. Al presionar *Iniciar*, el programa decide al azar qué jugador comienza.
4. Para cada jugador se calculan todos los caminos posibles aplicando la función de transición del AFND, y se separan los caminos ganadores (aquellos que llegan al estado final).
5. Se guardan cuatro archivos de texto: todos los caminos y los caminos ganadores de cada jugador.
6. Se elige al azar un camino ganador para cada jugador; si un jugador no tiene ninguno, se le asigna un camino cualquiera.
7. Cada dos segundos ambos jugadores avanzan una casilla por su ruta. El tablero anima el movimiento, la bitácora registra cada paso y se resuelven las colisiones (cuando ambos jugadores intentan ocupar la misma casilla).
8. El juego termina cuando un jugador alcanza su estado final o cuando ya no quedan movimientos; después, el tablero regresa a la posición inicial para una nueva partida. El botón *Detener* interrumpe la simulación en cualquier momento.

3.3 Estructura del programa

El código está organizado en once archivos de Java, uno por clase o interfaz. La clase principal, `Chess`, coordina el juego y se apoya en las demás, que se dividen en tres grupos: el modelo (la lógica del autómata y de los jugadores), la vista (los componentes de la ventana) y las utilidades. La Tabla 2 resume la responsabilidad de cada una.

Tabla 2: Clases del programa y su responsabilidad.

Clase	Grupo	Responsabilidad
`Chess`	Principal	Crea la ventana y coordina la partida: valida o genera las cadenas, ejecuta el AFND, elige las rutas y controla los turnos, las colisiones y el fin del juego.
`NfaSimulator`	Modelo	Motor del AFND: función de transición y cálculo de todos los caminos y de los caminos ganadores.
`SimulationResult`	Modelo	Guarda el resultado de una simulación: todos los caminos y los caminos ganadores.
`Player`	Modelo	Datos de cada jugador: estados inicial y final, color, cadena, caminos, ruta actual y posición. Elige su ruta al azar.
`ChessPanel`	Vista	Dibuja el tablero, las metas, las fichas, las animaciones y los indicadores de choque y de ganador.
`ControlPanel`	Vista	Controles de entrada: modo, cadenas, longitud automática y botones *Iniciar* y *Detener*.
`InfoPanel`	Vista	Barra de estado con el turno, el número de movimiento, el modo y el ganador.
`LogPanel`	Vista	Bitácora con todos los eventos de la partida.
`PathFileWriter`	Utilidad	Escribe los caminos en archivos de texto dentro de la carpeta `computing`.
`GameConfig`	Utilidad	Constantes: tamaño del tablero, longitud máxima de las cadenas (12) y colores.
`BoardStateProvider`	Utilidad	Interfaz con la que el tablero consulta las posiciones y el ganador sin depender de `Chess`.

Esta separación permite calcular los caminos sin depender de la ventana: el motor recibe la cadena y los estados inicial y final, y devuelve las rutas encontradas. El componente del tablero consulta las posiciones y el estado del ganador mediante una interfaz. El programa está escrito en Java (JDK 8 o superior) y utiliza las bibliotecas estándar: Swing y AWT para la ventana, listas para conservar los caminos y conjuntos para los estados siguientes.

3.4 Interfaz gráfica

Al ejecutar el programa se abre una ventana titulada “Tablero”, dividida en cuatro zonas (Figura 3):

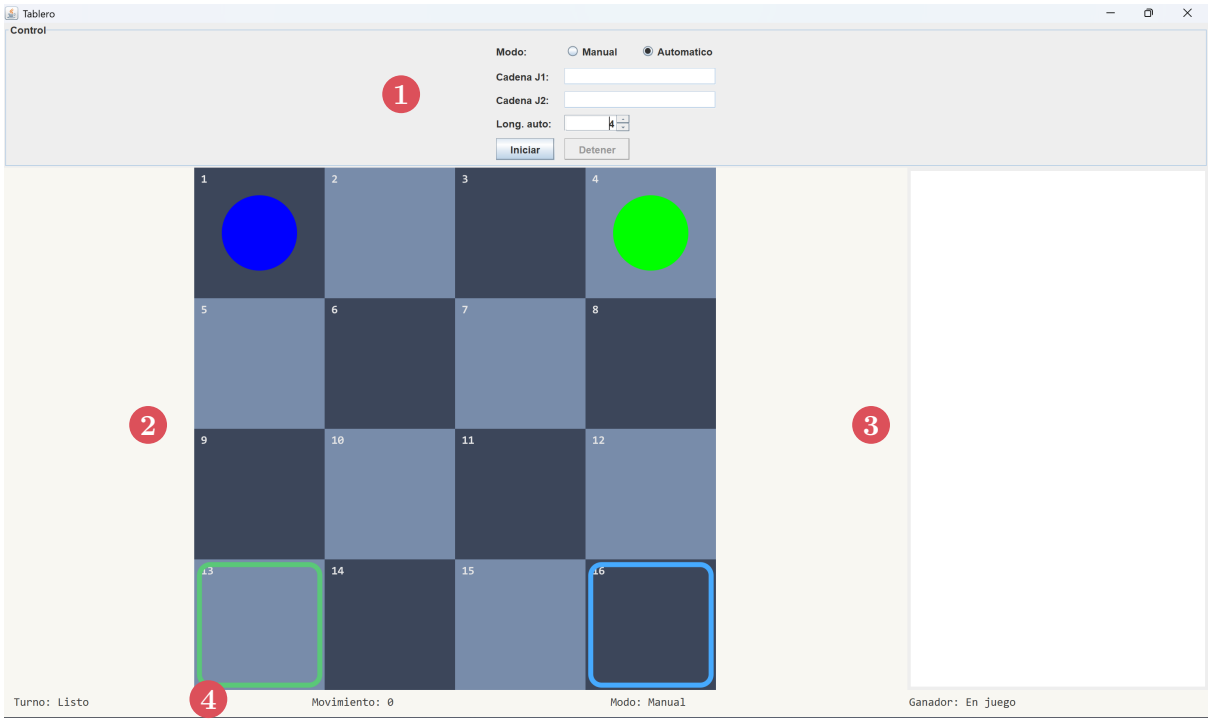


Figura 3: Ventana del programa antes de iniciar una partida: (1) panel de control, (2) tablero, (3) bitácora y (4) barra de estado.

1. **Panel de control** (parte superior). Permite elegir el modo (*Manual* o *Automático*), escribir la cadena de cada jugador (*Cadena J1* y *Cadena J2*), elegir la longitud de las cadenas aleatorias (*Long. auto*, de 3 a 12 símbolos) e iniciar o detener la simulación. Los campos de texto solo se habilitan en modo manual y el selector de longitud solo en modo automático. Mientras una partida está en curso se deshabilitan todos los controles excepto el botón *Detener*.

2. **Tablero** (centro). Muestra las 16 casillas numeradas con dos tonos alternados. La casilla meta de cada jugador se marca con un recuadro de su color: azul claro en la casilla 16 (meta de J1) y verde en la casilla 13 (meta de J2). Las fichas son un círculo azul (J1) y uno verde (J2). El tablero se adapta al tamaño de la ventana y siempre se mantiene centrado.

3. **Bitácora** (derecha). Registra en orden todos los eventos de la partida: qué jugador comienza, las cadenas generadas en modo automático, el número de caminos calculados, la posición de las fichas en cada movimiento, las colisiones, las redirecciones, las cesiones de paso y el final del juego. Los mismos mensajes se imprimen también en la consola.

4. **Barra de estado** (parte inferior). Muestra de quién es el turno (o `Listo` cuando no hay una partida en curso), el número de movimiento, el modo de juego y el campo de ganador.

Además de mostrar las posiciones, el tablero tiene tres efectos visuales:

- **Animación**: cada vez que las piezas cambian de casilla, se deslizan hasta su nuevo lugar en lugar de aparecer de golpe (Sección 3.5.9).

- **Choque**: cuando se detecta una colisión, se dibuja durante un segundo un recuadro rojo semitransparente con la palabra "Choque".
- **Ganador**: cuando un jugador llega a su meta, su ficha se rodea con un anillo de su color y aparece la palabra "GANADOR" sobre la casilla durante tres segundos; después, las fichas regresan a sus posiciones iniciales.

3.5 Explicación de los algoritmos utilizados

3.5.1 Función de transición

El método `nextStates` de la clase `NfaSimulator` implementa la función de transición δ: recibe el estado actual y un símbolo, y devuelve el conjunto de estados siguientes.
Explicación del algoritmo:

1. Convierte el número de estado (1–16) en fila y columna (0–3) con las fórmulas de la sección 2.2.
2. Si el símbolo es r, agrega las casillas de arriba $(q-4)$, abajo $(q+4)$, izquierda $(q-1)$ y derecha $(q+1)$.
3. Si el símbolo es b, agrega las cuatro diagonales: arriba-izquierda $(q-5)$, arriba-derecha $(q-3)$, abajo-izquierda $(q+3)$ y abajo-derecha $(q+5)$.
4. Antes de agregar cada casilla verifica, con la fila y la columna, que el movimiento no salga del tablero. Por ejemplo, desde la casilla 4 no se puede ir a la derecha, aunque $4+1=5$ sea un número de casilla válido.
5. Devuelve los estados en un conjunto (`HashSet`), por lo que no hay repetidos.

3.5.2 Cálculo de todos los caminos del AFND

El método `simulate` de `NfaSimulator` es la base del programa: dada la cadena de un jugador, su estado inicial y su estado final, calcula todos los caminos que puede seguir el autómata y separa los ganadores. Equivale a construir, nivel por nivel, el árbol de cómputo del AFND (Figura 4).
Explicación del algoritmo:

1. Se inicia con un único camino que contiene solo el estado inicial.
2. Para cada símbolo de la cadena, en orden:
   a) Para cada camino existente se toma su último estado y se obtienen sus estados siguientes con la función de transición.
   b) Por cada estado siguiente se crea una copia del camino con ese estado agregado al final.
   c) Los caminos nuevos reemplazan a los anteriores.
3. Al terminar la cadena, todos los caminos tienen $n+1$ estados. Los que terminan en el estado final se guardan, además, como caminos ganadores.

4. El resultado se devuelve como un objeto `SimulationResult`, que contiene la lista de todos los caminos y la de los caminos ganadores.

Los últimos estados de todos los caminos forman exactamente el conjunto $\hat{\delta}(q_0, w)$, y la cadena es aceptada si alguno de ellos es el estado final. Por ejemplo, para el Jugador 1 con la cadena *bbb* se obtienen 9 caminos y solo uno es ganador (Figura 4); por lo tanto, $\hat{\delta}(1, bbb) = \{6, 8, 14, 16\}$ y, como contiene al 16, la cadena es aceptada por A_1.

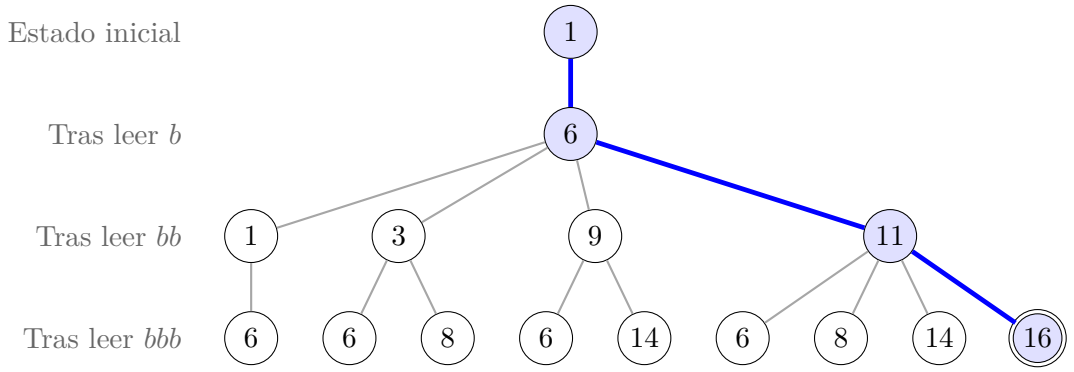


Figura 4: Árbol de cómputo del AFND del Jugador 1 para la cadena *bbb*. Cada rama de la raíz a una hoja es un camino; de los 9 caminos, solo $1 \to 6 \to 11 \to 16$ (resaltado) es ganador. Las hojas forman el conjunto $\hat{\delta}(1, bbb) = \{6, 8, 14, 16\}$.

Cada camino se ramifica en tantos caminos como opciones tenga su último estado, así que el número de caminos crece de forma exponencial con la longitud de la cadena: cada símbolo puede multiplicarlo hasta por cuatro. Esto determina el tiempo y la memoria que necesita el programa, y es la razón del límite de 12 símbolos (Sección 4.6).

3.5.3 Generación de los archivos de caminos

Una vez calculados los caminos, la clase `PathFileWriter` los guarda en la carpeta `computing`, que se crea automáticamente si no existe. Se generan cuatro archivos:

- `caminost1.txt`: todos los caminos del Jugador 1.
- `caminosg1.txt`: los caminos ganadores del Jugador 1.
- `caminost2.txt`: todos los caminos del Jugador 2.
- `caminosg2.txt`: los caminos ganadores del Jugador 2.

Cada línea del archivo es un camino, escrito como la sucesión de sus estados separados por espacios; por ejemplo, la línea `1 6 11 16` corresponde al camino ganador $1 \to 6 \to 11 \to 16$. Los archivos se sobrescriben en cada partida, así que siempre corresponden a las últimas cadenas simuladas.

3.5.4 Selección de rutas y del turno inicial

Cuando el usuario presiona *Iniciar*, el programa decide al azar qué jugador comienza y lo anota en la bitácora (por ejemplo, `Jugador 1 comienza`). Después de calcular los caminos, cada jugador elige la ruta que seguirá durante la partida:

1. Si tiene caminos ganadores, elige uno al azar.

2. Si no tiene ninguno (por ejemplo, porque su cadena tiene un número impar de símbolos r), se le asigna al azar un camino cualquiera del conjunto total. Así participa en la partida y puede estorbar al otro jugador, aunque su ruta no termine en su casilla final. Si esa ruta pasa por la meta antes de consumir toda la cadena, también puede ganar, porque la partida revisa la posición después de cada paso.

Como $\delta(q, a)$ nunca es vacío, el conjunto total de caminos nunca está vacío y siempre es posible asignarle una ruta a cada jugador.

3.5.5 Avance de la simulación y manejo de colisiones

La partida avanza con un temporizador de Swing (`javax.swing.Timer`) que ejecuta el método `nextMove` cada dos segundos. En cada movimiento *ambas* piezas avanzan una casilla a lo largo de su ruta; el turno no indica quién se mueve, sino quién tiene prioridad si las dos piezas quieren ocupar la misma casilla. La Figura 5 resume el proceso.
Explicación del algoritmo:

1. Incrementa el índice del movimiento actual. Si ya no quedan posiciones en las rutas, la partida termina sin ganador.

2. Obtiene la siguiente casilla de cada jugador según su ruta.

3. Verifica si hay colisión, es decir, si ambas casillas son la misma.

   a) Si hay colisión, el jugador que tiene el turno conserva su movimiento y el otro intenta redirigirse a una ruta alternativa (Sección 3.5.6).

   b) Si no es posible redirigirlo, ese jugador cede el paso: permanece en su casilla durante este movimiento y, en el siguiente, continúa con la posición que le corresponde en su ruta. El índice de movimiento avanza para ambos: el movimiento cedido no se repite.

4. Actualiza las posiciones de ambos jugadores, registra el movimiento en la bitácora (por ejemplo, `Movimiento 3: J1 en 6, J2 en 11`) e inicia la animación.

5. Verifica si algún jugador ha alcanzado su estado final; si es así, termina la partida (Sección 3.5.7).

6. Alterna el turno entre los jugadores solo si no hubo colisión; si la hubo, el turno se conserva.

7. Actualiza el tablero y la barra de estado.

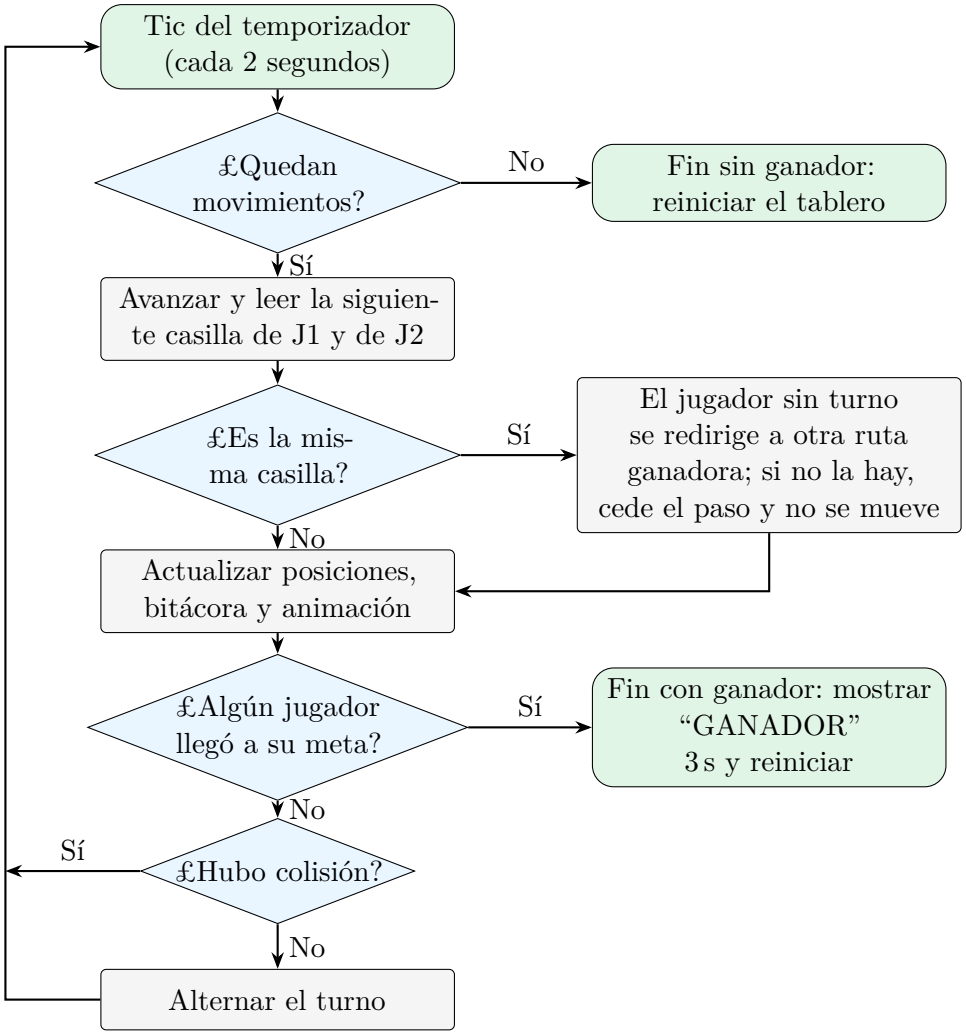


Figura 5: Diagrama de flujo de un movimiento de la simulación (método `nextMove`). Si hubo colisión, el turno no se alterna.

3.5.6 Redirección de jugadores

El método `redirectPlayer` busca un camino alternativo para el jugador que debe ceder cuando ocurre una colisión en el movimiento k.

Explicación del algoritmo:

1. Recorre los caminos ganadores del jugador, en el orden en que fueron calculados.
2. Descarta los caminos que no coinciden con la ruta actual en todas las posiciones anteriores a la colisión (de la 0 a la $k-1$); así, el cambio de ruta es consistente con lo que el jugador ya recorrió.
3. De los restantes, toma el primero cuya casilla en la posición k sea distinta de la de su ruta actual, es decir, que evite la casilla de la colisión.
4. Si lo encuentra, ese camino se convierte en la nueva ruta del jugador y la bitácora muestra, por ejemplo, `Jugador 2 redirigido a nueva ruta`. Si no lo encuentra, la bitácora muestra `Jugador 2 no puede moverse, cede el paso`.

Como la nueva ruta también es un camino ganador, el jugador redirigido conserva la posibilidad de llegar a su meta. Un jugador sin caminos ganadores nunca puede redirigirse, por lo que en caso de colisión siempre cede el paso.

3.5.7 Fin de la partida

Después de cada movimiento se revisa si algún jugador ocupa su casilla final:

- Si J1 está en la casilla 16, gana; si no, y J2 está en la 13, gana J2. Como J1 se revisa primero, si ambos llegan a su meta en el mismo movimiento se declara ganador a J1.
- Si hay ganador, se detiene el temporizador, la bitácora muestra `Jugador 1 ha ganado!` y `Jugador 1 ha llegado a su posicion final!` (o los mensajes equivalentes de J2), y el tablero resalta al ganador durante tres segundos. Después, las fichas regresan a sus posiciones iniciales y la bitácora indica `Jugadores regresados a posiciones iniciales`.
- Si se agotan los movimientos sin que nadie llegue a su meta, la bitácora muestra `Juego terminado - No hay ganador` y el tablero se reinicia de inmediato.
- En cualquier momento, el botón *Detener* interrumpe la simulación (`Simulacion detenida por el usuario`) y regresa las fichas a su posición inicial.

En todos los casos se vuelven a habilitar los controles, de modo que se puede iniciar otra partida sin cerrar la ventana.

3.5.8 Validación de las entradas

En el modo manual, antes de calcular los caminos se verifica que las cadenas cumplan las siguientes reglas; si alguna falla, el motivo aparece en la bitácora y la partida no inicia:

1. Ninguna cadena puede estar vacía.
2. Ninguna cadena puede tener más de 12 símbolos.
3. Solo se permiten los caracteres r y b (las mayúsculas se convierten a minúsculas).
4. Ambas cadenas deben tener la misma longitud, ya que en cada movimiento las dos piezas avanzan al mismo tiempo.

En el modo automático no hace falta esta validación: el programa genera dos cadenas aleatorias de la longitud elegida, en las que cada símbolo tiene la misma probabilidad de ser r o b, y la longitud siempre se mantiene entre 3 y 12. El límite de 12 símbolos (la constante `MAX_INPUT_LENGTH` de `GameConfig`) reduce el costo de enumerar todos los caminos; su justificación se analiza en la Sección 4.6.

3.5.9 Animación de los movimientos

Para que el recorrido se pueda seguir con la vista, la clase `ChessPanel` no dibuja las fichas directamente en su nueva casilla, sino que las desliza durante 250 milisegundos. Al iniciar la animación se guardan la fila y la columna de origen y de destino de cada ficha, y un segundo temporizador redibuja el tablero cada 16 milisegundos (unos 60 cuadros por segundo). En cada cuadro, la posición de la ficha se calcula por interpolación lineal:

$$\text{posición}(t) = \text{origen} + (\text{destino} - \text{origen}) \cdot \text{mín}\left(1, \frac{t}{250\ \text{ms}}\right), \tag{7}$$

donde t es el tiempo transcurrido desde el inicio del movimiento. La fórmula se aplica por separado a la fila y a la columna, por lo que en un movimiento diagonal la ficha avanza en línea recta sobre la diagonal. El tamaño de las casillas se recalcula cada vez que se dibuja el tablero a partir del tamaño de la ventana, y las fichas miden el 58 % del ancho de una casilla.

3.6 Uso del programa

En modo manual se selecciona *Manual*, se escribe una cadena de la misma longitud en cada campo y se presiona *Iniciar*. Por ejemplo, con *bbb* para ambos jugadores se calcula un camino ganador por jugador y las piezas pueden llegar a sus metas en tres movimientos. El recorrido se reproduce con el temporizador también en este modo: la entrada es manual, pero no se mueve cada ficha mediante botones independientes.

En modo automático se selecciona *Automático*, se ajusta *Long. auto* y se presiona *Iniciar*. El programa genera las dos cadenas, registra sus resultados y reproduce una partida. Para realizar otra se vuelve a presionar *Iniciar*; el reinicio del tablero no genera partidas consecutivas por sí solo. *Detener* cancela el recorrido y devuelve las piezas al inicio.

Los archivos de caminos se crean en la carpeta `computing`, dentro de la carpeta desde la que se ejecuta el programa.

3.7 Particularidades del funcionamiento actual

El cálculo del AFND y la partida tienen criterios diferentes. Un camino del autómata es ganador si termina en la meta después de leer la cadena completa; la partida termina en cuanto una ficha ocupa su meta, incluso antes del último símbolo. Los archivos contienen las rutas calculadas antes de jugar y no se modifican por las colisiones ni por una victoria anticipada.

La detección de colisiones compara las dos casillas de destino. No comprueba el intercambio de posiciones ni todos los casos en que una pieza permanece quieta. Además, después de ceder el paso se continúa con el siguiente índice de la ruta prevista, sin recalcular el recorrido desde la posición real. Esto puede producir un desplazamiento que no sea adyacente o que no corresponda al símbolo siguiente; por ello, el recorrido animado tras una cesión puede diferir de un camino válido del AFND.

Las capturas de victoria muestran que la barra inferior puede conservar `Ganador: En juego` aunque el tablero ya resalte al ganador. La bitácora y el anillo con la palabra "GANADOR" permiten identificar el resultado de esas partidas.

4 Resultados

Las capturas disponibles documentan varias partidas en modo automático con cadenas de distintas longitudes. A continuación se presentan esas capturas, junto con la explicación de lo que muestra la bitácora en cada caso, y el contenido de los archivos de caminos generados.

4.1 Inicio de una partida

En la Figura 6 se eligió el modo automático con cadenas de longitud 4. El programa decidió que comenzara el Jugador 1 y generó las cadenas *rrrb* (J1) y *rbrb* (J2). La bitácora muestra que J1 tiene 46 caminos posibles, pero ninguno ganador, mientras que J2 tiene 38 caminos, de los cuales 2 son ganadores: $4 \to 3 \to 6 \to 10 \to 13$ y $4 \to 8 \to 11 \to 10 \to 13$. Esto coincide con la condición de la Sección 2.5: *rrrb* tiene tres símbolos *r*, así que es imposible que J1 gane con ella, mientras que *rbrb* tiene dos.

Durante la partida, el panel de control queda deshabilitado (salvo el botón *Detener*) y la barra de estado indica el número de movimiento y el turno, que ya pasó a J2 porque el primer movimiento no tuvo colisión.

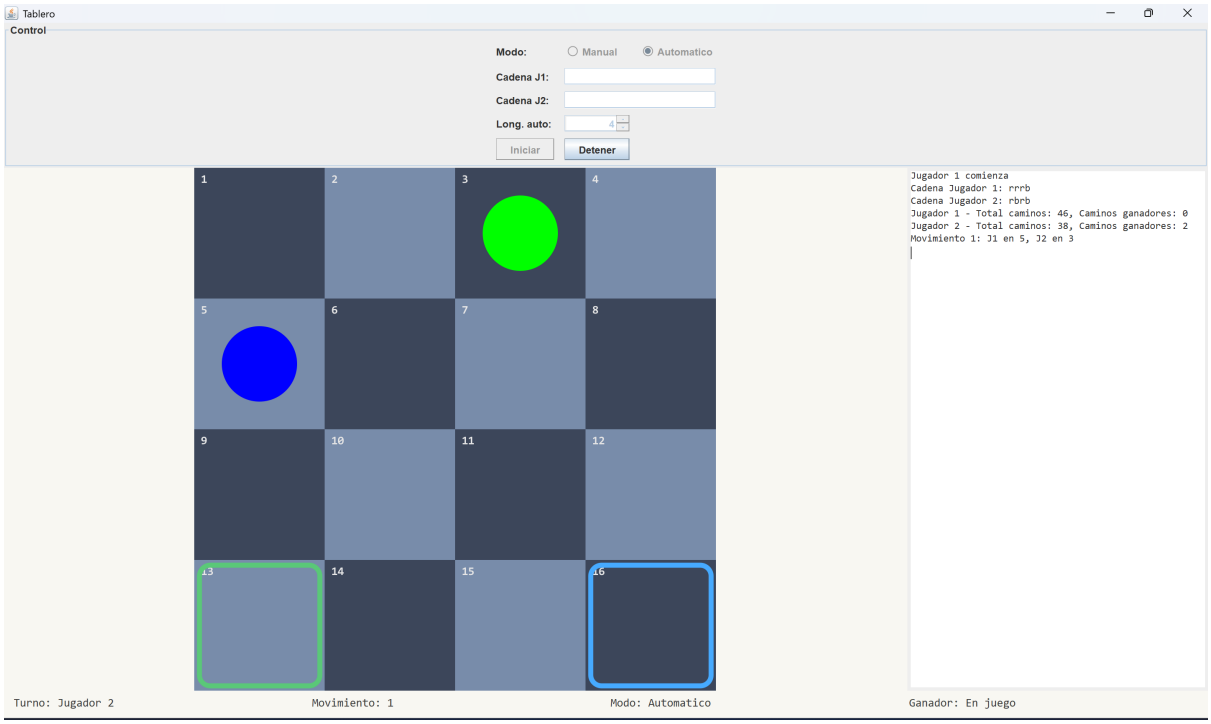


Figura 6: Partida en modo automático con cadenas de longitud 4, después del primer movimiento.

4.2 Victoria del Jugador 2

La Figura 7 muestra el final de esa misma partida. Como J1 no tenía caminos ganadores, recorrió un camino cualquiera ($1 \to 5 \to 1 \to 2 \to 5$), mientras que J2 siguió su camino ganador $4 \to 3 \to 6 \to 10 \to 13$. En el cuarto movimiento, J2 llegó a la casilla 13: el tablero lo resalta con un anillo verde y la palabra “GANADOR”, y la bitácora registra

`Jugador 2 ha ganado!`. Tres segundos después, las fichas regresaron a sus posiciones iniciales.

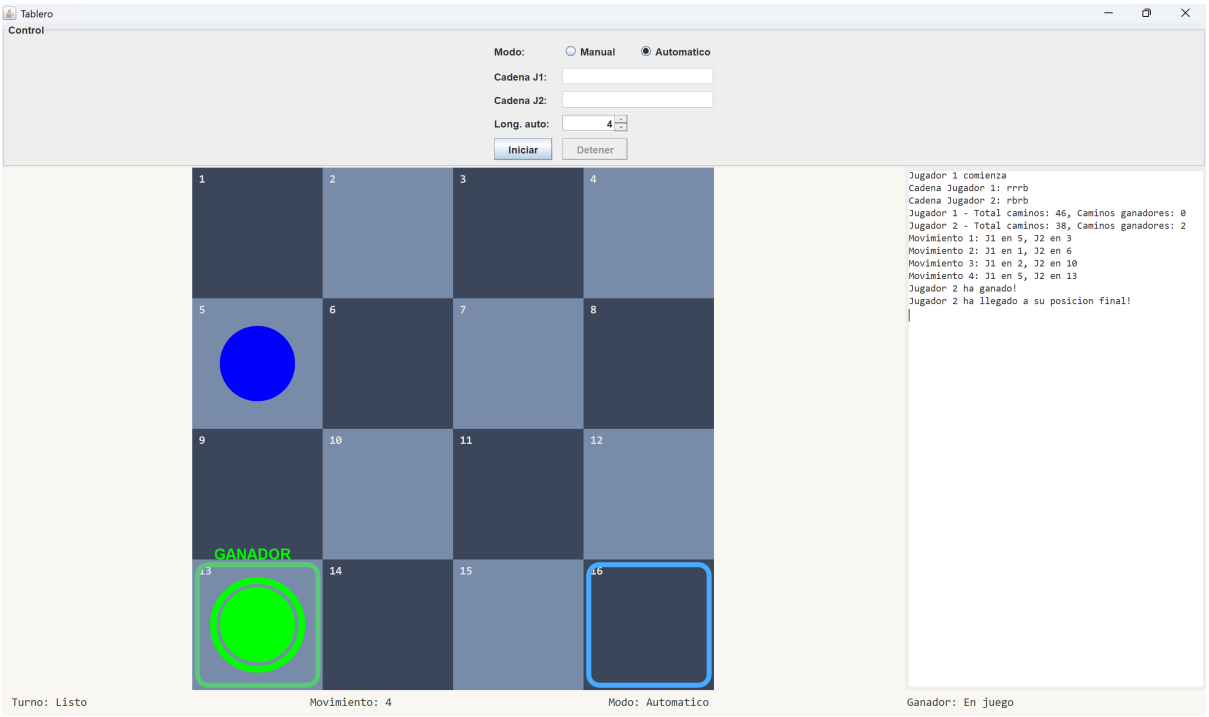


Figura 7: El Jugador 2 llega a la casilla 13 en el cuarto movimiento y gana la partida.

4.3 Colisión y cesión del paso

En la siguiente partida (Figura 8) se usaron cadenas de longitud 7: *rbbrrbb* para J1 y *rbbbrbr* para J2. Ambas tienen tres símbolos r, así que ningún jugador tiene caminos ganadores (834 caminos cada uno, ninguno ganador) y los dos recorren rutas cualesquiera. En el quinto movimiento ambos intentaron entrar a la casilla 2: J1 desde la casilla 1 y J2 desde la 3. Como el turno era de J1, este conservó su movimiento; J2 intentó redirigirse, pero al no tener caminos ganadores no encontró ninguna ruta alternativa y cedió el paso, quedándose en la casilla 3. El tablero marca la casilla 2 con el recuadro rojo de “Choque” y la barra de estado conserva el turno de J1, ya que en un movimiento con colisión el turno no cambia.

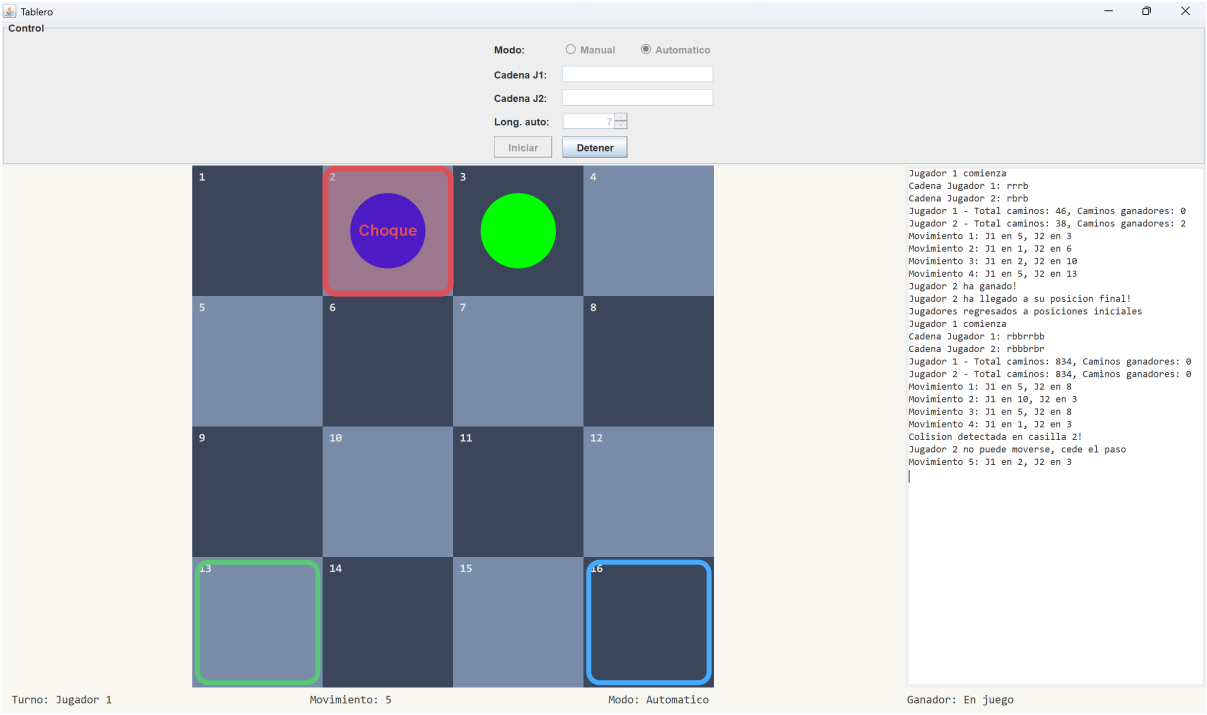
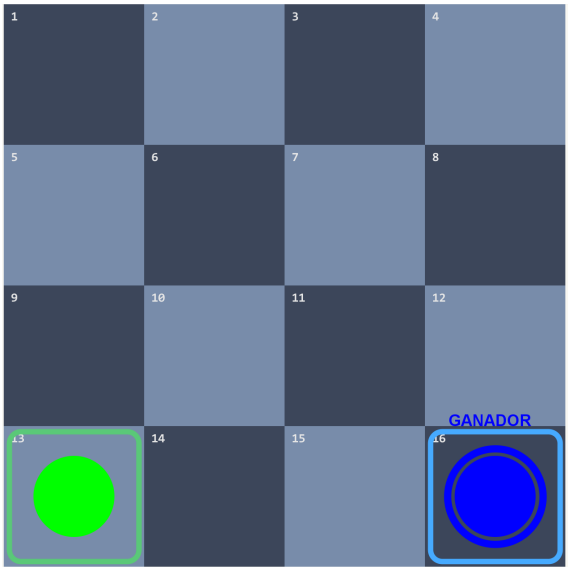


Figura 8: Colisión en la casilla 2 durante el quinto movimiento: el Jugador 2 no tiene una ruta alternativa y cede el paso.

4.4 Redirección y victoria del Jugador 1

La Figura 9 muestra la bitácora de varias partidas consecutivas. En la penúltima, con las cadenas *brrbrb* y *rrbrrr*, ningún jugador tenía caminos ganadores (las dos cadenas tienen un número impar de símbolos r), así que la partida terminó después del sexto movimiento con el mensaje `Juego terminado - No hay ganador`. En la última, con cadenas de longitud 9, comenzó J2; J1 tenía 10 790 caminos (762 ganadores) y J2, 4 622 caminos (336 ganadores). La Tabla 3 reconstruye esa partida a partir de la bitácora.



(a) Ambas fichas llegan a su meta; se resalta a J1.

```
Jugador 2 no puede moverse, cede el paso
Movimiento 6: J1 en 7, J2 en 3
Movimiento 7: J1 en 2, J2 en 11
Juego terminado - No hay ganador
Jugadores regresados a posiciones iniciales
Jugador 2 comienza
Cadena Jugador 1: brrbrb
Cadena Jugador 2: rrbrrr
Jugador 1 - Total caminos: 318, Caminos ganadores:
0
Jugador 2 - Total caminos: 486, Caminos ganadores:
0
Movimiento 1: J1 en 6, J2 en 3
Movimiento 2: J1 en 5, J2 en 7
Movimiento 3: J1 en 1, J2 en 4
Movimiento 4: J1 en 6, J2 en 8
Movimiento 5: J1 en 2, J2 en 12
Movimiento 6: J1 en 5, J2 en 11
Juego terminado - No hay ganador
Jugadores regresados a posiciones iniciales
Jugador 2 comienza
Cadena Jugador 1: rbrbrrrbr
Cadena Jugador 2: brbbbbbrb
Jugador 1 - Total caminos: 10790, Caminos
ganadores: 762
Jugador 2 - Total caminos: 4622, Caminos ganadores:
336
Movimiento 1: J1 en 5, J2 en 7
Movimiento 2: J1 en 2, J2 en 6
Movimiento 3: J1 en 6, J2 en 11
Movimiento 4: J1 en 9, J2 en 16
Movimiento 5: J1 en 10, J2 en 11
Colision detectada en casilla 6!
Jugador 2 redirigido a nueva ruta
Movimiento 6: J1 en 6, J2 en 16
Movimiento 7: J1 en 7, J2 en 11
Movimiento 8: J1 en 12, J2 en 10
Movimiento 9: J1 en 16, J2 en 13
Jugador 1 ha ganado!
Jugador 1 ha llegado a su posicion final!
```

(b) Detalle de la bitácora de las partidas.

Figura 9: Detalles de una misma captura. En la última partida, el Jugador 2 es redirigido después de una colisión y el Jugador 1 gana en el noveno movimiento.

Tabla 3: Reconstrucción de la última partida de la Figura 9, con las cadenas *rbrbrrrbr* (J1) y *brbbbbbrb* (J2). La columna *Turno* indica qué jugador tenía prioridad en cada movimiento.

Mov.	Turno	Símbolos (J1, J2)	J1	J2	Observación
–	–	–	1	4	Posiciones iniciales; comienza J2.
1	J2	r, b	5	7	
2	J1	b, r	2	6	
3	J2	r, b	6	11	
4	J1	b, b	9	16	
5	J2	r, b	10	11	
6	J1	r, b	6	16	Choque en la casilla 6; J2 es redirigido.
7	J1	r, b	7	11	El turno no cambió por la colisión.
8	J2	b, r	12	10	
9	J1	r, b	16	13	Ambos llegan a su meta; gana J1.

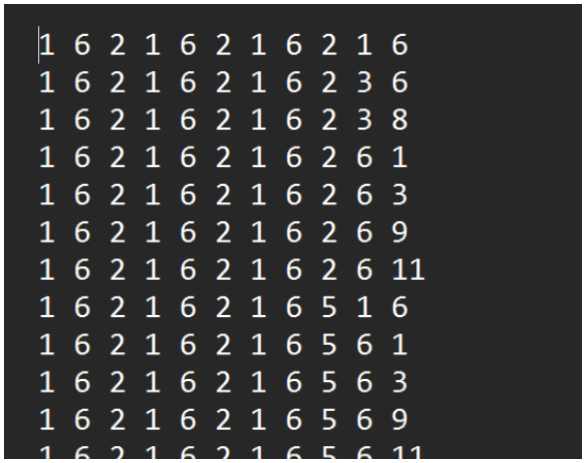
En el sexto movimiento, los dos jugadores querían entrar a la casilla 6. El turno era de J1, así que J2 buscó entre sus caminos ganadores uno que coincidiera con lo que ya había recorrido ($4 \to 7 \to 6 \to 11 \to 16 \to 11$) y que no pasara por la casilla 6 en ese momento; lo encontró y fue redirigido a la casilla 16. Cada paso de la tabla respeta la función de transición: por ejemplo, en el movimiento 4, J1 lee b y pasa de 6 a 9, una de sus diagonales. En el noveno movimiento ambos jugadores llegaron a su meta al mismo tiempo, pero, como la condición de J1 se revisa primero, se le declaró ganador.

4.5 Archivos de caminos generados

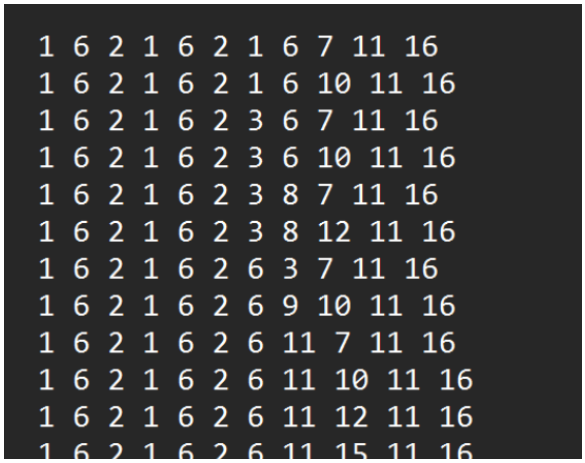
Los archivos disponibles corresponden a las cadenas *brrbrrbrrb* (J1) y *brbbrbbbbb* (J2). Cada una tiene 10 símbolos y un número par de movimientos r. La Tabla 4 resume los archivos generados y la Figura 10 muestra sus primeras líneas.

Tabla 4: Archivos generados para las cadenas *brrbrrbrrb* (J1) y *brbbrbbbbb* (J2).

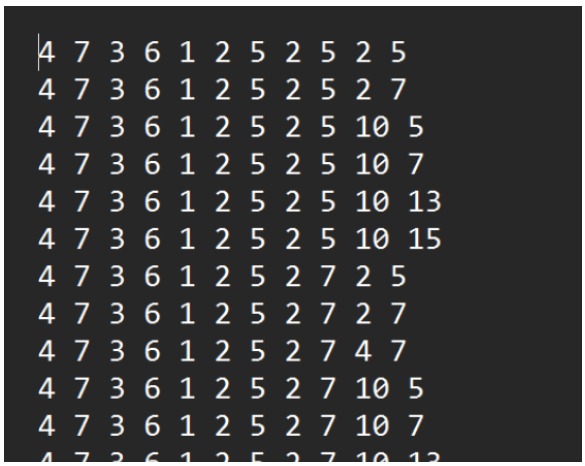
Archivo	Contenido	Caminos	Tamaño
`caminost1.txt`	Todos los caminos del Jugador 1	28 250	746 kB
`caminosg1.txt`	Caminos ganadores del Jugador 1	2 036	57 kB
`caminost2.txt`	Todos los caminos del Jugador 2	12 102	327 kB
`caminosg2.txt`	Caminos ganadores del Jugador 2	882	25 kB



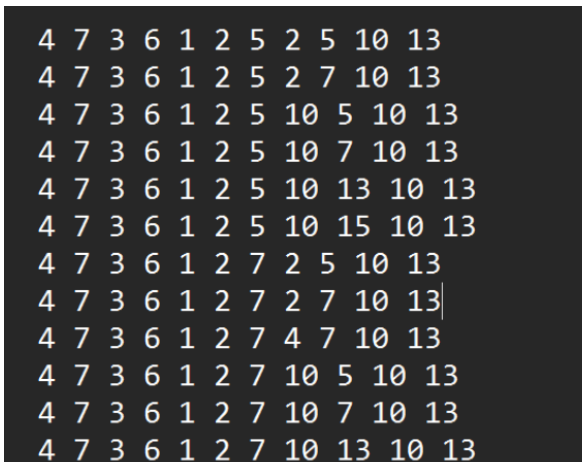
(a) `caminost1.txt`



(b) `caminosg1.txt`



(c) `caminost2.txt`



(d) `caminosg2.txt`

Figura 10: Primeras líneas de los archivos de caminos. Cada línea es un camino de 11 estados.

Cada línea tiene 11 estados: el inicial más uno por cada uno de los 10 símbolos. Todas las líneas de los archivos del Jugador 1 empiezan en 1 y las del Jugador 2 en 4; en los archivos de caminos ganadores, además, todas terminan en la meta del jugador (16 o 13). Cada par de estados consecutivos respeta la función de transición: por ejemplo, en la primera línea de `caminost1.txt`, $1 \to 6$ es un movimiento diagonal (b), $6 \to 2$ y $2 \to 1$ son ortogonales (r), y así sucesivamente, siguiendo la cadena $brrbrrbrrb$.

4.6 Crecimiento del número de caminos

Si se permitieran cadenas largas, el cálculo y el almacenamiento de las rutas requerirían cada vez más tiempo y memoria. La causa es el crecimiento exponencial del número de caminos. La Tabla 5 muestra cuántos caminos tiene el Jugador 1 con las cadenas r^n (solo movimientos ortogonales) y b^n (solo diagonales); se comprobó que, para $n \leq 12$, r^n es la cadena de longitud n con más caminos posibles. Por la simetría del tablero, los valores son los mismos para el Jugador 2.

Tabla 5: Número de caminos del Jugador 1 según la longitud n de la cadena, comparado con la cota 4^n.

n	Caminos con r^n	Caminos con b^n	Cota 4^n
3	18	9	64
6	602	169	4 096
9	20 378	3 025	262 144
10	65 946	7 921	1 048 576
11	213 402	20 736	4 194 304
12	690 586	54 289	16 777 216
20	8 305 351 066	119 814 916	$\approx 1.1 \times 10^{12}$
100	$\approx 5.3 \times 10^{50}$	$\approx 3.3 \times 10^{41}$	$\approx 1.6 \times 10^{60}$

Con cada símbolo r adicional el número de caminos se multiplica por aproximadamente 3.24, y con cada b, por aproximadamente 2.62. Con la longitud máxima permitida (12 símbolos), el peor caso son 690 586 caminos, y el archivo `caminost1.txt` ocuparía alrededor de 20 MB; en cambio, con 20 símbolos serían más de 8 300 millones de caminos, y con 100 símbolos, del orden de 10^{50}, una cantidad imposible de almacenar en memoria o de escribir en un archivo. Por esta razón, el programa limita las cadenas a 12 símbolos.

Cabe destacar que el crecimiento exponencial se debe a que el programa debe *enumerar* todos los caminos para escribir los archivos. Para decidir únicamente si una cadena es aceptada, bastaría con llevar el conjunto de estados actuales $\hat{\delta}(q_0, w)$, que nunca tiene más de 16 elementos, lo que toma un tiempo proporcional a la longitud de la cadena.

5 Conclusión

Considero que ha sido una de las prácticas más retadoras a la que me he enfrentado en toda mi estancia en ESCOM, ya que el programa tenía muchas cosas importantes que tener en cuenta. Quizá de lo más complejo fue implementar el modo gráfico para visualizar el tablero en ejecución. Me gustó la práctica, ya que se pusieron a prueba los conocimientos adquiridos en las clases, y los que he adquirido en toda la carrera. Me gustó utilizar estructuras de datos, algoritmos y un poco de matemáticas para la elaboración del programa.

En esta versión, organizar el código en clases con responsabilidades separadas (el motor del autómata, el modelo de cada jugador, la escritura de archivos y los componentes de la ventana) facilitó agregar el panel de control, la bitácora, la barra de estado y las animaciones. También pude comprobar en la práctica que el número de caminos de un AFND crece de forma exponencial con la longitud de la cadena, por lo que fue necesario limitar las cadenas a 12 símbolos. Sin duda es una práctica muy completa para el entendimiento de los autómatas finitos no determinísticos.

6 Referencias bibliográficas

[1] John E. Hopcroft and Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, Addison-Wesley Publishing, Reading Massachusetts, 1979.

[2] Láminas del curso de autómatas de la Universidad de Stanford (J. D. Ullman): `http://infolab.stanford.edu/~ullman/ialc/spr10/slides/fa2.pdf`